CHAPTER 1
INTRODUCTION

Scientific and technical documents often contain complex mathematical expressions embedded within large volumes of unstructured text. Extracting and understanding these equations manually presents several challenges for students, researchers, and professionals. Firstly, there is a significant risk of misinterpretation due to ambiguous notation, inconsistent variable definitions, and lack of contextual clarity. Equations written across documents may reuse symbols with different meanings, leading to confusion and incorrect analysis. Additionally, manually identifying relationships between equations requires substantial effort, as dependencies are not explicitly represented, making it difficult to track how variables and formulas are interconnected.

Secondly, the manual analysis of equations is time-consuming and inefficient. Researchers and learners spend considerable effort reading documents, identifying equations, and understanding their structure and meaning. This process diverts attention from core activities such as research, experimentation, and knowledge discovery. Furthermore, equations are often distributed across multiple pages and formats (PDFs, images, DOCX), making retrieval and cross-referencing cumbersome. The lack of structured representation limits the ability to perform automated reasoning, validation, or large-scale analysis.

Moreover, traditional document formats do not provide interactive capabilities for exploring equations. Retrieving specific equations, understanding their dependencies, or detecting inconsistencies requires manual inspection, which is both slow and error-prone. Storage and accessibility also pose challenges, as documents are static and lack machine-readable structure. This restricts advanced use cases such as graph-based visualization, semantic analysis, and intelligent querying. Additionally, ensuring correctness and consistency across equations is difficult without automated validation mechanisms, leading to potential errors in research or learning outcomes.

1.1 Problem Statement:

The project aims to develop an intelligent web-based application that can extract, analyze, and represent equations from scientific documents in a structured and interactive manner. EquateAI focuses on transforming unstructured document content

into machine-readable representations, enabling efficient analysis and reasoning over equations.

Transitioning from manual equation analysis to an automated, AI-assisted platform represents a significant advancement in handling scientific documents. The proposed system provides a centralized digital environment where equations are detected, parsed, and analyzed systematically. This platform allows users to upload documents and retrieve structured outputs such as equation lists, variable roles, dependency graphs, and knowledge graphs. Unlike traditional approaches, this system is not limited by document format or physical accessibility, enabling users to access and analyze content from anywhere.

Furthermore, the adoption of automated parsing and semantic analysis improves both accuracy and efficiency. The system minimizes human errors by using deterministic parsing techniques and validation mechanisms to ensure consistency in equation interpretation. It also identifies issues such as missing variables, multiple definitions, and dependency cycles, which are difficult to detect manually. By integrating AI-based explanations and document-aware chat functionality, the platform enhances understanding and usability, allowing users to interact with equations in a more intuitive manner.

Additionally, the system provides advanced visualization capabilities through graph representations. Dependency graphs and knowledge graphs enable users to explore relationships between variables and equations interactively. These features support better comprehension, analysis, and decision-making, making the system valuable for educational, research, and professional applications.

1.2 Objectives:

The main intention of this project is to develop an AI-assisted system for extracting, analyzing, and visualizing equations from scientific documents. The objectives of EquateAI are as follows:

1. Develop a user-friendly web interface that enables users to upload documents and interact with extracted equation data efficiently.
2. Implement automated equation detection and parsing mechanisms to convert

unstructured text into structured representations such as Abstract Syntax Trees.

3. Perform semantic analysis to classify variables and identify relationships such as defined, used, and undefined variables.
4. Detect inconsistencies including multiple definitions, symbolic conflicts, and missing variables to improve data reliability.
5. Generate dependency graphs and knowledge graphs to visualize relationships between equations and variables.
6. Integrate AI-based explanation and chat features to assist users in understanding equations and document content.
7. Ensure system performance, scalability, and responsiveness to handle multiple documents and large datasets efficiently.
8. Provide secure access and data handling mechanisms to protect user-uploaded documents and processed information.

1.3 Scope of the Project:

The scope of EquateAI includes several key functionalities aimed at improving the analysis and understanding of scientific documents. The system serves as a centralized platform for uploading, processing, and analyzing documents containing mathematical expressions. It supports multiple input formats such as PDFs, DOCX files, and images, enabling flexible document ingestion.

The platform provides automated extraction and structuring of equations, including parsing into syntax trees and identification of variable roles. Users can access detailed information about each equation, including its structure, dependencies, and semantic meaning. Additionally, the system offers interactive graph-based visualizations such as dependency graphs and knowledge graphs, allowing users to explore relationships between equations and variables in an intuitive manner.

Advanced features such as conflict detection, missing variable identification, and dependency cycle analysis are included to enhance the quality and reliability of extracted information. The system also incorporates AI-assisted explanation and chat functionalities, enabling users to query equations and receive contextual responses based on document content.

The design of the system emphasizes scalability and performance, ensuring efficient processing of large documents and datasets. Integration capabilities allow the

system to be extended with additional tools or services if required. Security measures are implemented to protect user data, including controlled access, safe file handling, and privacy considerations.

Overall, the project focuses on providing an end-to-end solution for document-centric equation analysis, combining deterministic processing with AI-assisted features to enhance usability and analytical capabilities.

1.4 Motivation:

In existing workflows, equations within scientific documents are analyzed manually, which leads to inefficiencies, inconsistencies, and difficulty in understanding complex relationships. Researchers and students often spend significant time identifying equations, interpreting their meanings, and tracking dependencies across documents. This manual process increases the likelihood of errors, such as incorrect variable interpretation or overlooked inconsistencies, which can impact the quality of analysis and research outcomes.

Additionally, the lack of structured representation limits the ability to perform advanced operations such as automated validation, dependency tracking, and graph-based visualization. Traditional document formats do not support interactive exploration, making it challenging to gain deeper insights from the content. Furthermore, accessing and analyzing equations across multiple documents is time-consuming and requires significant manual effort.

EquateAI is motivated by the need to simplify and enhance this process by providing an intelligent, automated system for equation analysis. By converting unstructured content into structured representations and enabling interactive exploration through graphs and AI-assisted explanations, the system reduces manual effort and improves accuracy. It empowers users to focus on understanding and applying knowledge rather than spending time on repetitive and error-prone tasks. Ultimately, the project aims to make scientific documents more accessible, interpretable, and actionable for a wide range of users

.

CHAPTER 2
SYSTEM ANALYSIS

System analysis is an essential phase in the development lifecycle of any software project, as it establishes the foundation for successful design and implementation. It involves a detailed study of the project objectives, functional expectations, technical requirements, and operational limitations. Through careful system analysis, the overall scope of the project becomes clear, enabling developers and stakeholders to understand the workflow, identify dependencies, evaluate possible risks, and propose effective technical solutions.

For EquateAI, system analysis is particularly important because the project combines document processing, equation extraction, semantic interpretation, graph construction, and AI-assisted reasoning within a single full-stack application. The system must support multiple document formats, process mathematical content reliably, store structured outputs, and present them through an interactive web interface. Therefore, analyzing the current challenges of scientific document interpretation and defining a suitable proposed solution are necessary steps in building an efficient and scalable platform

2.1 System Requirement Specification:

In the area of intelligent document analysis and web-based scientific tools, several developments have highlighted the importance of usability, automation, and secure processing. Modern web applications increasingly focus on user-centered design, responsive interaction, and workflow automation to improve accessibility and reduce manual effort. In systems handling technical or research-oriented data, the ability to process, retrieve, and visualize complex information has become a major requirement.

Studies and practical applications in web systems emphasize that usability plays a critical role in adoption. A platform intended for researchers, students, and technical users must provide an interface that is easy to navigate while still exposing advanced analytical features. Similarly, performance and responsiveness are important, particularly when processing large files such as scientific PDFs or documents containing many equations.

Security is another major consideration. Since uploaded documents may contain

sensitive research material, the system must ensure controlled access, secure data storage, and safe handling of files. At the same time, the architecture should remain flexible enough to integrate emerging technologies such as AI-based assistants, local large language model inference, and graph-based document exploration.

Overall, the requirement specification for EquateAI is shaped by the need to combine document ingestion, equation parsing, semantic reasoning, and user interaction into one cohesive web-based environment. The system must be efficient, accurate, scalable, and understandable for end users

2.1.1 Existing System:

The existing approach for understanding equations in scientific documents is largely manual and fragmented. Researchers, students, and technical professionals usually read through PDFs, reports, or scanned documents and identify equations by visual inspection. This process is slow, repetitive, and often error-prone. Equations are not stored in a structured form, and relationships between variables are rarely made explicit.

In many cases, users rely on static documents, screenshots, handwritten notes, or generic OCR tools that only extract plain text without understanding the mathematical structure. Such methods do not provide Abstract Syntax Trees, semantic classification of variables, or graph-based representations of dependencies. As a result, identifying undefined variables, repeated definitions, symbolic inconsistencies, or circular relationships becomes difficult and time-consuming.

The current workflow also lacks centralized analysis. Scientific content remains locked inside documents, making retrieval and cross-referencing inefficient. Existing systems may offer text extraction, but they generally do not combine parsing, semantic analysis, graph construction, and AI-based explanation in an integrated way. Therefore, the existing system is characterized by limited automation, weak interpretability, and insufficient support for advanced document-centric reasoning

2.1.2 Proposed System:

The proposed system, EquateAI, is an advanced web-based platform developed to automate the extraction and analysis of equations from scientific documents. It provides

a centralized environment where users can upload documents in formats such as PDF, DOCX, and images, and obtain structured outputs including detected equations, syntax trees, variable roles, dependency graphs, knowledge graphs, and AI-assisted explanations.

The system is designed with a Fast API backend and a Next.js frontend. The backend coordinates the complete processing pipeline, including document reading, preprocessing, heuristic equation detection, lexical parsing, semantic analysis, dependency analysis, conflict checking, and graph generation. The frontend offers an interactive and user-friendly interface through which users can upload documents, browse extracted equations, visualize graph structures, and interact with the AI chat assistant.

The proposed system reduces manual effort and improves consistency by converting unstructured mathematical content into machine-readable and navigable data. It enables users to locate equations quickly, understand variable relationships, identify missing or conflicting definitions, and inspect equation dependencies visually. The integration of local LLM support further enhances the platform by providing natural-language explanations and contextual chat responses. Overall, the proposed system aims to overcome the limitations of traditional manual workflows by offering a comprehensive, efficient, and intelligent solution for scientific equation analysis.

Requirement Analysis:

Analysis involves breaking down a complete system into its individual components in order to understand its structure, purpose, and operation. Design, on the other hand, focuses on creating a suitable plan for arranging those components into an effective and usable system. System analysis and design together include a range of technical activities intended to understand requirements, evaluate alternatives, and guide implementation successfully.

The tasks involved in system analysis include:

1. **Understanding the application:** Studying the purpose of the system, expected users, supported workflows, and the type of scientific content it must process.
2. **Planning:** Defining the architecture, technologies, development flow, and implementation strategy required for the project.
3. **Scheduling:** Organizing development stages such as backend API creation, frontend integration, parsing modules, graph visualization, and testing.

4. **Developing candidate solutions:** Exploring possible methods for document reading, equation detection, parsing, semantic analysis, and graph representation.
5. **Performing trade studies:** Comparing rule-based techniques, symbolic analysis approaches, and AI-assisted methods to identify suitable design choices.
6. **Performing cost-benefit analysis:** Evaluating implementation complexity, performance, maintainability, and usability benefits of the selected technologies.
7. **Recommending alternative solutions:** Proposing the most appropriate architecture and workflow for document-centric equation reasoning.
8. **Presenting the system:** Demonstrating the practical value of the platform for researchers, students, and technical users.
9. **Supervising, deploying, and maintaining the system:** Ensuring smooth implementation, resolving issues, and supporting future improvements and updates.

Software Requirement Specification:

The Software Requirements Specification (SRS) represents the final outcome of the analysis process and serves as a blueprint for the development of the system. It defines the functional requirements, expected behavior, performance needs, design limitations, and validation expectations of the software. In the case of EquateAI, the SRS outlines the requirements for document upload, equation extraction, AST generation, semantic analysis, graph creation, data persistence, and AI-based assistance.

The SRS also acts as a communication bridge between stakeholders and developers by providing a clear statement of system expectations. It guides the design, implementation, testing, and maintenance of the application. By defining the scope and operational behavior in advance, it helps ensure that the final system is aligned with user needs and technical objectives.

2.1.3 Software and Hardware Required:

Designing a software system and successfully implementing it are two different aspects of development. Successful implementation depends not only on software tools, but also on hardware support and user readiness. The system is intended to run on a personal computer or laptop capable of supporting both frontend and backend development, local database usage, document processing, and local LLM execution.

The following hardware components are typically sufficient:

HARDWARE used in the system:

The following components are used:

- Personal Computer (PC) / Laptop
- AMD Ryzen 7 processor
- 16 GB RAM or more
- Sufficient storage for project files, uploaded documents, and local models
- Internet connection for setup, package installation, and updates

SOFTWARE used in the system:

- Microsoft Windows 11
- Visual Studio Code
- Python
- Node.js
- SQLite or PostgreSQL
- Ollama (for local LLM integration)
- Tesseract OCR (for image-based text extraction)

Technologies used in the project:

- Python
- FastAPI
- Next.js
- React
- TypeScript / JavaScript
- Tailwind CSS
- SQL Alchemy
- SymPy
- NetworkX
- SQLite / PostgreSQL

Non-functional Requirements

Usability: The user is provided with an interactive interface through which documents can be uploaded, processed, and explored. The extracted equations, graphs, and AI-generated explanations are displayed in a way that supports ease of understanding and

efficient navigation.

Security and support: The application must ensure safe file handling, controlled API access, and secure storage of processed data. User-uploaded documents should be protected from unauthorized access, and the platform should support stable and reliable operation.

Maintainability: The project is organized into clear modules such as agents, services, routes, schemas, and frontend components. This modular structure simplifies debugging, updating, and future enhancements.

2.2 Technology tools and programming languages used:

Visual Studio Code:

Visual Studio Code is the main development environment used for building EquateAI. It supports both frontend and backend development with features such as debugging, terminal integration, syntax highlighting, and extension support. It is used to manage the FastAPI backend, Next.js frontend, and project configuration files efficiently.

Python:

Python is the core backend language of EquateAI. It is used for document reading, preprocessing, equation detection, lexical parsing, semantic analysis, graph construction, symbolic reasoning, and API development. Its rich ecosystem of libraries makes it highly suitable for analytical and scientific applications.

FastAPI:

FastAPI is the backend framework used to create high-performance REST APIs. It manages document upload, processing, equation explanation, graph retrieval, and chat services. Its automatic validation and API documentation features make development more efficient and maintainable.

Next.js:

Next.js is the main frontend framework used to build the web interface of EquateAI. It supports structured routing, responsive design, and smooth integration with backend APIs. It is used for pages such as upload, document analysis, graph views, and chat.

React:

React is used to create reusable and interactive frontend components. It helps in building dynamic interfaces such as dashboards, equation cards, graph viewers, and chat panels. Its component-based design improves maintainability and scalability.

TypeScript / JavaScript:

TypeScript and JavaScript are used in the frontend for logic, API communication, and user interaction. TypeScript improves reliability through type safety, while JavaScript supports dynamic behavior and compatibility.

Tailwind CSS:

Tailwind CSS is used for frontend styling. It enables rapid UI development with utility-based classes and helps create responsive, clean, and consistent layouts across the application.

SQLAlchemy and Database Systems:

SQLAlchemy is used for database management and object-relational mapping in the backend. It stores documents, equations, variables, dependencies, conflicts, missing variables, and chat records. SQLite is the default database, with PostgreSQL support available if needed.

SymPy:

SymPy is used for symbolic mathematics and equation reasoning. It supports simplification, comparison, solving, differentiation, and integration, mainly helping with symbolic conflict detection and analytical processing.

NetworkX:

NetworkX is used for graph construction and analysis. It supports dependency graph generation and cycle detection, which are important for understanding relationships between equations and variables.

Ollama:

Ollama provides local large language model integration for equation explanation, document-aware chat, and text-to-equation assistance. It allows EquateAI to include AI-powered features without depending on cloud-based services.

PyMuPDF, python-docx, Pillow, and pytesseract:

These libraries are used for document ingestion:

- **PyMuPDF** for PDF reading
- **python-docx** for DOCX extraction
- **Pillow** for image processing
- **pytesseract** for OCR-based text extraction

Together, they allow the system to process multiple document formats in a unified pipeline.

CHAPTER 3
SYSTEM DESIGN

This chapter focuses on the design of the EquateAI system, which is described through its data flow design, architecture, module interactions, and database structure. The system design transforms the requirements and analysis developed in earlier stages into a practical software structure that can be implemented efficiently. It defines how data is captured from uploaded documents, processed through various analytical stages, stored in the database, and presented to the user through the frontend interface.

System Data Flow Design:

A data flow diagram is a graphical representation of how data moves through a system. It shows the input provided to the application, the transformations performed by different processes, the generated outputs, and the storage locations used for persistence. Unlike a control-flow diagram, a data flow design focuses mainly on the movement and transformation of information rather than execution timing or sequencing details. In the case of EquateAI, the data flow design helps illustrate how uploaded scientific documents are converted into structured equation data, semantic information, graph representations, and AI-assisted outputs.

The data flow design of this application includes document upload, file storage, text extraction, preprocessing, equation detection, parsing, semantic analysis, graph generation, persistence to the database, and frontend retrieval for interactive visualization and chat-based reasoning

3.1 System Architecture:

3.1.1 Document Processing and Analysis Module:

The main processing module of EquateAI is structured into multiple connected stages, each serving a specific function in the overall workflow. The first stage handles document upload and reading. Users can upload documents in PDF, DOCX, or image formats. These files are stored temporarily and then passed to the document reader, which extracts text content based on the file type.

The second stage focuses on preprocessing and equation detection. Extracted text is normalized to handle mathematical symbols, formatting irregularities, and noise lines. The system then identifies equation-like expressions using heuristic scoring and filtering methods. These candidate equations are passed to the lexer and parser components, which generate structured Abstract Syntax Trees (ASTs) for further analysis.

The third stage performs semantic and dependency analysis. Variables are identified and classified according to their roles, such as defined, used, or undefined. Dependency relationships between variables are then derived, allowing the system to construct dependency graphs and knowledge graphs. Additional checks are applied to detect missing variables, symbolic conflicts, multiple definitions, and dependency cycles. Together, these modules provide a complete analytical pipeline for converting unstructured scientific documents into structured, machine-readable, and interpretable outputs.

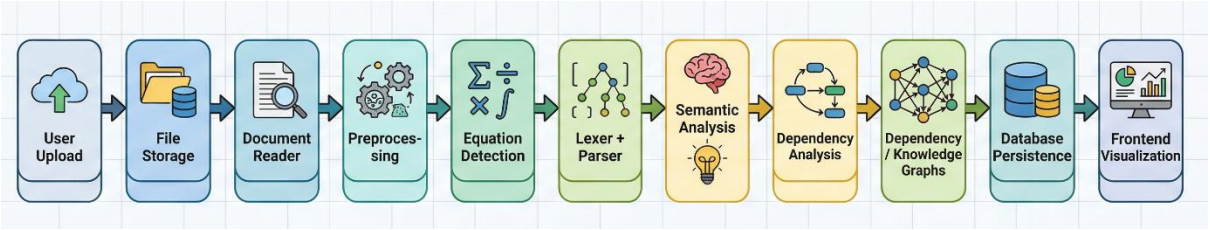

Figure 3.1a: Model for Document Processing and Analysis

Figure 3.1a illustrates the architectural model of the document processing and analysis workflow. This model provides a clear overview of how the system transforms uploaded files into meaningful structured data, helping users navigate the core analytical capabilities of the application.

3.1.2 AI Explanation and Chat Module:

Figure The AI interaction module provides natural-language assistance over the extracted equations and document context. This module supports equation explanation, text-to-equation assistance, and document-aware chat. Once equations and summaries have been generated and stored, the user can request explanations for a particular equation or interact with the system through a chat interface.

The backend retrieves relevant context from the processed document,

including extracted equations and associated structured information, and sends it to the local language model through Ollama. The generated response is then returned to the frontend and displayed to the user. This interaction enables the system to go beyond static extraction and provide an intelligent, assistive layer for better understanding the document content.

Figure 3.1b provides a visual representation of the AI explanation and chat flow within EquateAI. It shows the interaction between user requests, stored document context, local LLM processing, and response generation.

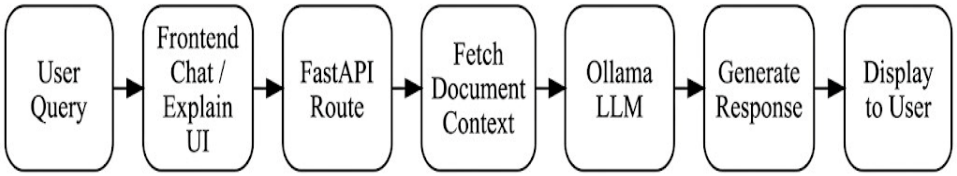

Figure 3.1b: Model for AI Explanation and Chat Module

3.2 Flow charts:

3.2.1 Uploading and Processing a New Document:

In **Figure 3.2a**, a detailed flow diagram illustrates the step-by-step process of uploading and processing a new scientific document in the system. The process begins when the user uploads a valid file through the frontend interface. The backend validates the file type and stores it in the configured upload directory. After successful validation, the system initiates document processing.

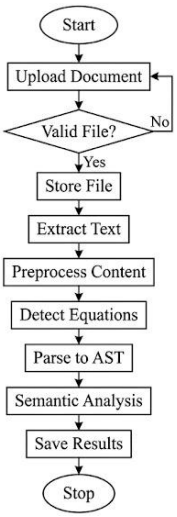

Figure 3.2a: Flow of uploading and processing of a new document.

The document reader extracts text from the file, after which preprocessing is applied to clean and normalize the content. Equation detection is then performed to identify equation-like expressions. These equations are parsed into structured syntax trees, semantically analyzed, and used to generate dependency and knowledge graph data. Finally, all processed results are stored in the database and made available for later retrieval and visualization. This flow ensures that document analysis is performed in a systematic and reliable manner.

3.2.2 Retrieving Existing Document Analysis:

The process of retrieving an existing document analysis begins by selecting or providing the identifier of a previously uploaded document. This step enables the system to fetch the stored details related to that document, including its summary, extracted equations, graph structures, conflict information, missing variables, and dependency issues.

By using the document identifier, the frontend can request relevant information from the backend through dedicated API routes. The backend then queries the database and returns the stored artifacts in a structured form. This approach gives users direct access to previously processed documents without repeating the entire analysis pipeline. **Figure 3.2b** represents the flow for searching and retrieving existing document analysis in EquateAI.

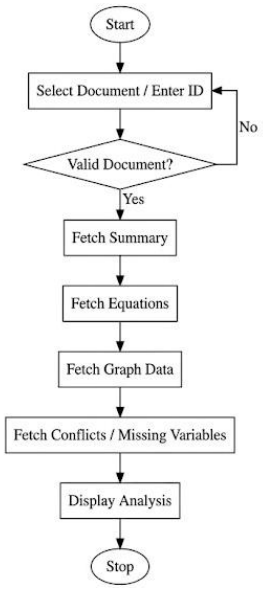

Figure 3.2b: Flow of retrieval of existing document analysis.

3.2.3 Equation Analysis and Graph Generation:

Figure 3.2c presents a detailed flow of equation analysis and graph generation. Once equation candidates are detected from the document, they are tokenized and parsed into AST structures. The semantic analyzer identifies the defined and used variables, and the dependency module constructs variable relationships from the parsed equations.

These relationships are then used to generate dependency graphs and knowledge graphs. At the same time, additional validation is performed to detect missing variables, conflicting symbolic definitions, and cyclic dependencies. This flowchart provides a clear roadmap of how the system transforms equations into graph-based representations that users can visualize and explore interactively.

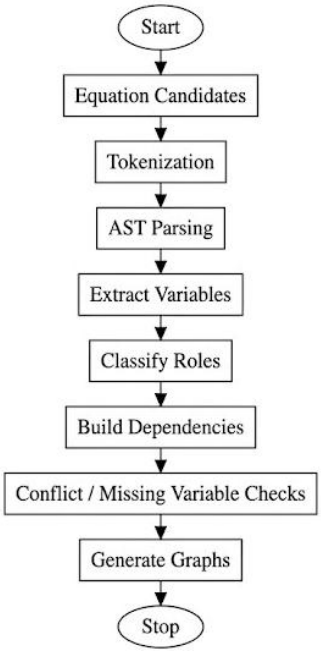

Figure 3.2c: Flow of equation analysis and graph generation.

3.3 Class Diagrams:

In a full-stack web-based application, a class diagram serves as a structural blueprint that depicts the major classes, entities, and their relationships within the system. It provides a visual representation of the backend and data architecture, showing how components interact to fulfill the functional requirements of the application.

For EquateAI, the class diagram includes the primary entities involved in the processing lifecycle, such as Document, Equation, Variable, Dependency, Conflict, MissingVariable, and ChatHistory. It also reflects the interaction between service classes,

agent modules, and API routes that coordinate the processing of uploaded files and the retrieval of structured outputs. This representation is useful for understanding how the application organizes information internally and how different components collaborate to perform tasks such as parsing, analysis, graph generation, and AI-assisted interaction.

3.4 Database Design:

A database design of EquateAI serves as the persistent storage layer for all analytical outputs generated during document processing. A database schema acts as a blueprint that defines the structure, organization, and relationships of the stored data. It specifies the tables used in the application, the columns within those tables, and the associations between records.

In EquateAI, the database stores information related to uploaded documents, extracted equations, semantic variables, dependencies, conflicts, missing variables, and chat history. The schema supports both operational functionality and later retrieval of processed results. This design allows users to revisit documents, inspect equation data, analyze graph structures, and continue chat-based interactions without having to repeat the document processing workflow.

The tables included in the database design are centered around the following entities:

- **Documents**
- **Equations**
- **Variables**
- **Dependencies**
- **Conflicts**
- **Missing Variables**
- **Chat History**

Each table contains attributes relevant to its role in the system. For example, the Documents table stores file name, type, path, upload date, and processing status. The Equations table stores the raw equation text, parsed AST, and associated document link. Variables and Dependencies tables capture semantic relationships, while Conflicts and Missing Variables tables record validation results. Chat History stores user-assistant interactions tied to a specific document context.

Figure 3.3 represents the database design of EquateAI and shows the tables and their relationships in the system.

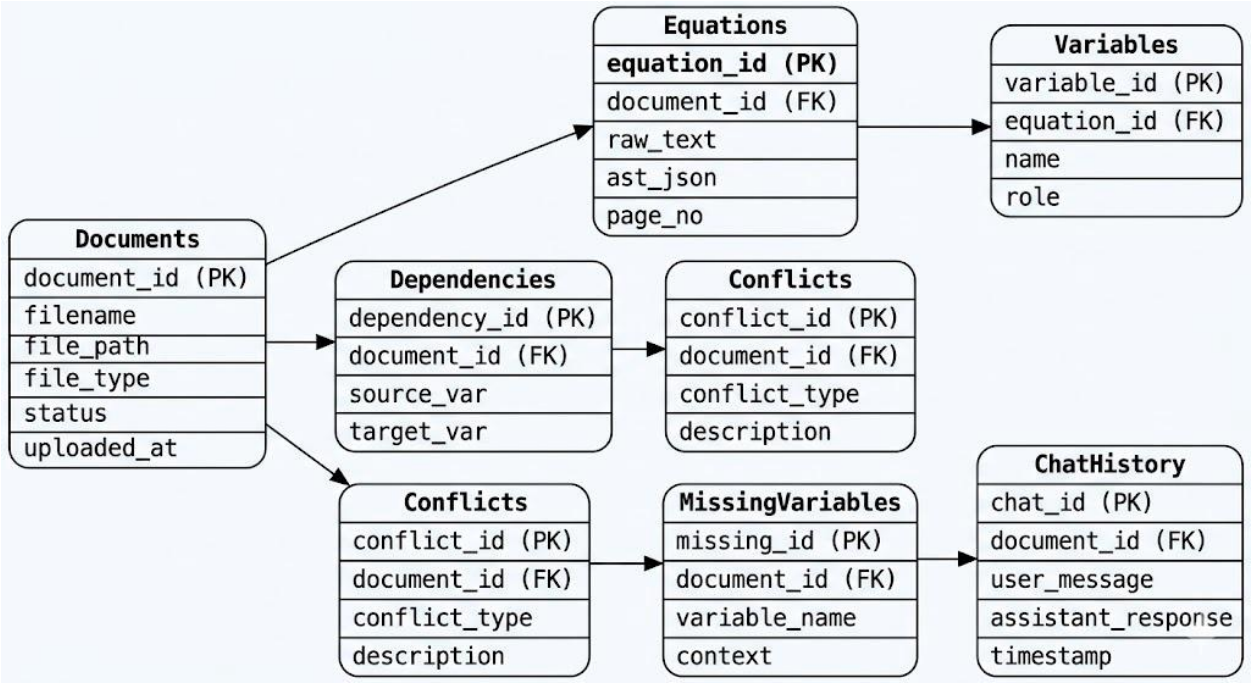

Figure 3.3 Database Design

CHAPTER 4
SYSTEM IMPLEMENTATION

The implementation of our project, **"Scientific Equation Parser & Analysis tool"** involves the development and integration of multiple modules such as document upload, text extraction, equation detection, parsing, semantic analysis, graph generation, database persistence, and AI-assisted explanation. The system is designed as a full-stack web application using a FastAPI backend and a Next.js frontend. Through the careful selection of technologies, structured modular design, and integration of deterministic analysis with local language model support, the implementation ensures efficient processing of scientific documents and meaningful equation-centric reasoning.

The backend is responsible for reading uploaded files, preprocessing extracted content, identifying equation-like expressions, converting them into structured forms, and storing the results for later retrieval. The frontend provides an interactive environment where users can upload documents, browse extracted equations, visualize dependency and knowledge graphs, and communicate with the AI assistant for explanations. Together, these components form a complete system that improves the accessibility and interpretability of mathematical content in scientific documents.

Project Repository: https://github.com/Gangula-Venkata-Rishitha/EquateAI

4.1 Setup Prerequisites:

To implement and run EquateAI successfully, both software and runtime prerequisites are required.

- **Python Environment:** Install Python and the required backend libraries such as FastAPI, SQLAlchemy, Pydantic, SymPy, NetworkX, PyMuPDF, Pillow, python-docx, and pytesseract.
- **Node.js Environment:** Install Node.js and npm to run the Next.js frontend application.
- **Database Support:** SQLite is used by default for local storage, while PostgreSQL can be configured if needed for larger-scale deployment.
- **OCR Support:** Tesseract OCR must be installed for extracting text from image-based documents.
- **LLM Support:** Ollama should be installed and running locally to enable equation

explanation and document-aware chat.

- **Development Tools:** IDEs such as Visual Studio Code can be used for development, debugging, and project management.
- **Operating System:** The project can be executed on Windows, Linux, or macOS, depending on dependency compatibility and local setup.

These prerequisites provide the foundation for executing both the backend processing pipeline and the frontend user interface smoothly.

4.2 Code Implementation of Accounts:

The primary input to EquateAI is a scientific document uploaded by the user. The system accepts multiple document formats such as PDF, DOCX, and images. Once uploaded, the file is stored in a configured upload directory, and its metadata is recorded in the database. This forms the starting point of the document analysis workflow.

After storage, the backend processing pipeline is triggered. The document reader module selects the appropriate extraction method depending on file type. PDF files are processed using PDF extraction tools, DOCX files are parsed through document libraries, and image-based files are handled using OCR support. The extracted text is then passed to the preprocessing module for normalization and cleanup before equation detection begins.

This integration of file handling, extraction, and preprocessing ensures that different document formats can be processed within a single unified workflow.

4.3 Data Processing and Optimization:

To prepare the extracted text for reliable analysis, several optimization and transformation steps are applied in sequence. The preprocessing stage handles normalization of mathematical symbols, whitespace cleanup, and removal of noisy text patterns such as headers or page numbers. This improves the quality of candidate equation detection.

The equation detection module then scans the cleaned text and assigns heuristic confidence scores to identify equation-like expressions. These detected candidates are passed to the lexical and parsing modules, which convert them into structured token streams and Abstract Syntax Trees (ASTs). Once structured, the semantic analyzer identifies variable roles, such as defined, used, and undefined variables.

Further optimization is performed by deriving dependencies between variables and building graph representations from these relationships. Additional checks such as symbolic conflict detection, missing variable analysis, and dependency cycle identification improve the reliability of the output. These processing stages collectively ensure that the document content is transformed from unstructured text into structured, analyzable data.

4.4 System Performance Analysis:

The performance of EquateAI can be understood in terms of processing flow, structured output generation, and user interaction efficiency.

- **Document Processing Efficiency:** The system supports end-to-end processing from upload to structured analysis without manual intervention.
- **Equation Parsing Capability:** The parser converts detected equations into structured AST representations, enabling deeper semantic analysis.
- **Semantic Reliability:** The system identifies defined, used, and undefined variables, helping users understand equation roles and detect inconsistencies.
- **Graph Generation:** Dependency graphs and knowledge graphs provide interactive visual exploration of variable and equation relationships.
- **AI Assistance:** The local language model integration improves user understanding by generating natural-language explanations and contextual responses.

The implementation is designed more as an application-oriented intelligent system than a benchmark-heavy research pipeline. Therefore, its performance is evaluated mainly through functional completeness, consistency of outputs, and usability of generated analytical artifacts.

CHAPTER 5
SYSTEM TESTING

Testing phase of the Software Development Life Cycle (SDLC) is one of the most important stages in ensuring that the developed system works correctly, reliably, and according to user requirements. During this phase, the developed modules are verified to confirm that they perform the intended functions and produce accurate outputs. Before testing begins, suitable test scenarios and use cases are identified so that the behavior of the software can be examined systematically.

In the case of EquateAI, testing is essential because the system combines multiple functionalities such as document upload, text extraction, equation detection, parsing, semantic analysis, graph generation, database storage, and AI-assisted explanation. Since the application contains several interconnected modules, all of them must be tested both individually and together to ensure consistency of operation. The purpose of system testing in this project is to validate the complete workflow, including user interaction, backend processing, data persistence, and output visualization.

5.1 Types of Testing:

The in a software system may occur at different stages of development, so the testing process must be performed at multiple levels. The basic levels of testing carried out for this system are **Unit Testing**, **Integration Testing**, **System Testing**, and **Acceptance Testing**.

Unit testing is performed on individual modules to verify that each component functions according to its design. Integration testing is then carried out by combining these modules and checking whether they interact correctly. System testing validates the complete software as a whole, while acceptance testing focuses on whether the system satisfies the intended requirements from the user's perspective.

5.1.1 Unit Testing:

Unit testing focuses on the smallest individual parts of the software, usually at the module or function level. The purpose of this testing is to identify errors within the boundaries of a specific component before it is integrated with other parts of the

application.

In EquateAI, unit testing was applied to the individual modules such as:

- document upload validation
- file type handling
- document reading for PDF, DOCX, and image files
- preprocessing and text normalization
- equation detection
- lexical parsing and AST generation
- semantic role assignment
- dependency analysis
- graph data generation
- conflict and missing-variable detection
- AI explanation and chat request handling

Each of these units was tested separately for correctness, error handling, and expected outputs. For example, the upload module was tested for valid and invalid file formats, the parser was tested for proper equation structure generation, and the semantic analyzer was tested for correct classification of variables. The unit testing process confirmed that the core modules were functioning correctly within their individual scope.

5.1.2 Integration Testing:

Integration testing is used to verify that the separately tested modules work correctly when combined into a single application. The purpose is to identify interface-related issues and confirm that data flows properly from one module to another.

In EquateAI, the individual units tested in the earlier phase were integrated into the complete document analysis workflow. For example, the document upload module was connected with file storage, the document reader was linked to preprocessing, and the parsing module was connected to semantic analysis and graph generation. Similarly, the processed data was connected to database persistence and frontend retrieval routes.

The interaction between backend routes, services, database models, and frontend components was tested to ensure consistency and correct information transfer. The integrated modules such as upload, process, summary retrieval, equation retrieval, graph visualization, and AI chat were verified together. The results showed that the modules

worked in coordination and met the required functional expectations.

5.2 Test Case:

EquateAI system supports multiple major operations, and corresponding test cases were used to verify them.

In the document processing module:

A user can upload a scientific document and process it for analysis. The test case includes the following steps:

- Login or access the application interface.
- Upload a valid document in PDF, DOCX, or image format.
- Trigger the document processing operation.
- Verify whether the text is extracted successfully.
- Check whether equations are detected and stored.
- Verify whether semantic analysis and graph generation are completed.

Confirm that the processed document summary and outputs are displayed correctly.

In the retrieval and analysis module:

Previously processed documents can be searched and analyzed again. The test case includes:

- Select an existing processed document or document ID.
- Retrieve summary information from the backend.
- Display extracted equations and parsed results.
- Fetch dependency graph and knowledge graph data.
- Check for missing variables, dependency issues, and conflicts.

In the AI explanation and chat module:

Users can request equation explanation or interact with the document-aware assistant. The test case includes:

- Select a processed equation.
- Request equation explanation.
- Verify response generation from the local LLM.
- Start a document-aware chat query.
- Check whether the response uses the relevant document context correctly.

These test cases ensured that the major workflows of the application were functioning as intended.

5.3 System Review and Maintenance:

Afte successful implementation and testing, system review and maintenance are necessary to ensure the continued effectiveness of the application. The following aspects are considered during review and maintenance:

- Comparison of the system's actual performance with the intended functional objectives.
- Evaluation of each module with respect to reliability, usability, and correctness.
- Correction of any errors discovered after practical usage.
- Updating and extending the system with new features and improvements.

A review is a process in which the software product is examined critically to determine whether it continues to meet the required standards. In EquateAI, review includes checking whether document processing remains stable, whether the parser handles expected equation forms correctly, whether graph outputs are meaningful, and whether AI-based assistance remains useful and relevant.

Maintenance is also an important activity in a web-based application. Common maintenance tasks may include fixing bugs, updating dependencies, improving parser rules, optimizing graph visualization, enhancing chat functionality, and extending support for more document types or equation formats. Regular maintenance ensures that the system remains secure, stable, and useful over time.

5.4 User Guide:

Users of the software should follow the guidelines below for proper operation of EquateAI:

- Make sure Python, Node.js, database support, OCR dependencies, and Ollama are properly configured.
- Upload only supported file formats such as PDF, DOCX, and valid image files.
- Verify that the backend and frontend servers are running before accessing the application.
- Check database and file storage availability for smooth document processing.
- Ensure that local LLM models are installed and active if AI explanation and chat features are to be used.

CHAPTER 6

RESULTS

The developed system, **EquateAI**, provides an interactive and intelligent platform for analyzing scientific documents. The results of the system demonstrate successful integration of document upload, equation extraction, semantic analysis, graph generation, and AI-based explanation. The following sections describe the major outputs of the system with respect to different interfaces and functionalities.

6.1 Home page:

homepage, as shown in Figure 6.1, acts as the entry point to the EquateAI system. It presents a clean and modern interface highlighting the core functionality of the application, which is AI-powered scientific equation analysis. Users are provided with a direct option to upload documents and begin processing.

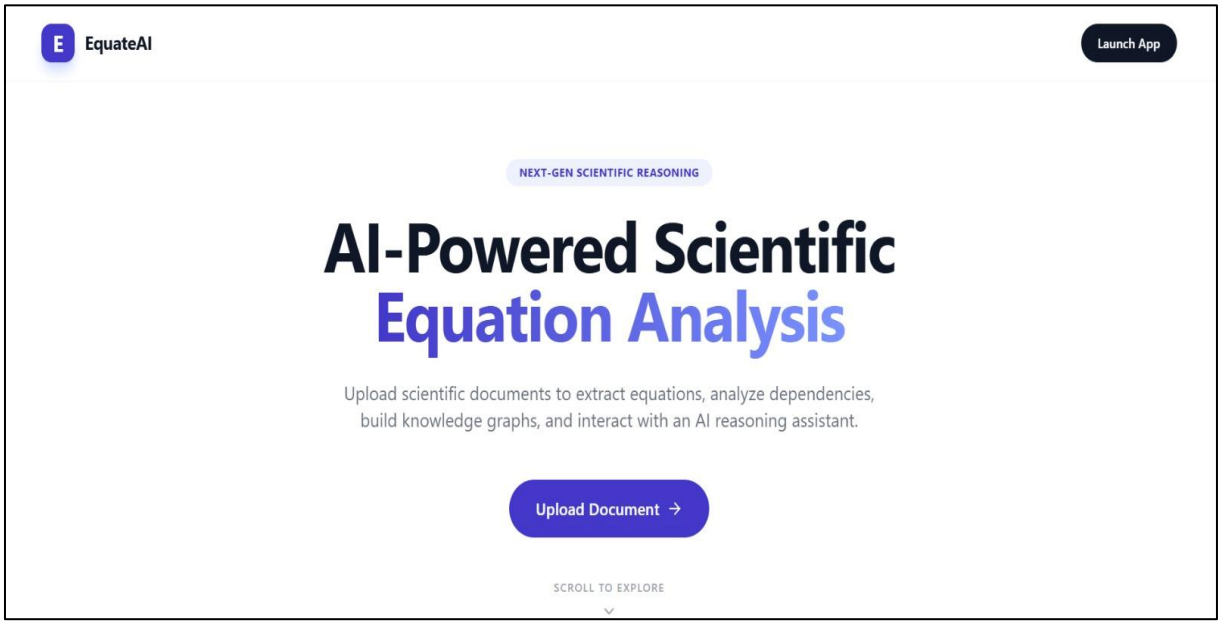

Figure 6.1: Home page

6.2 Document Upload Interface:

The upload interface, shown in Figure 6.2, allows users to submit scientific documents in multiple formats such as PDF, DOCX, and image files. Users can select a file and initiate processing using the “Upload & Process” option.

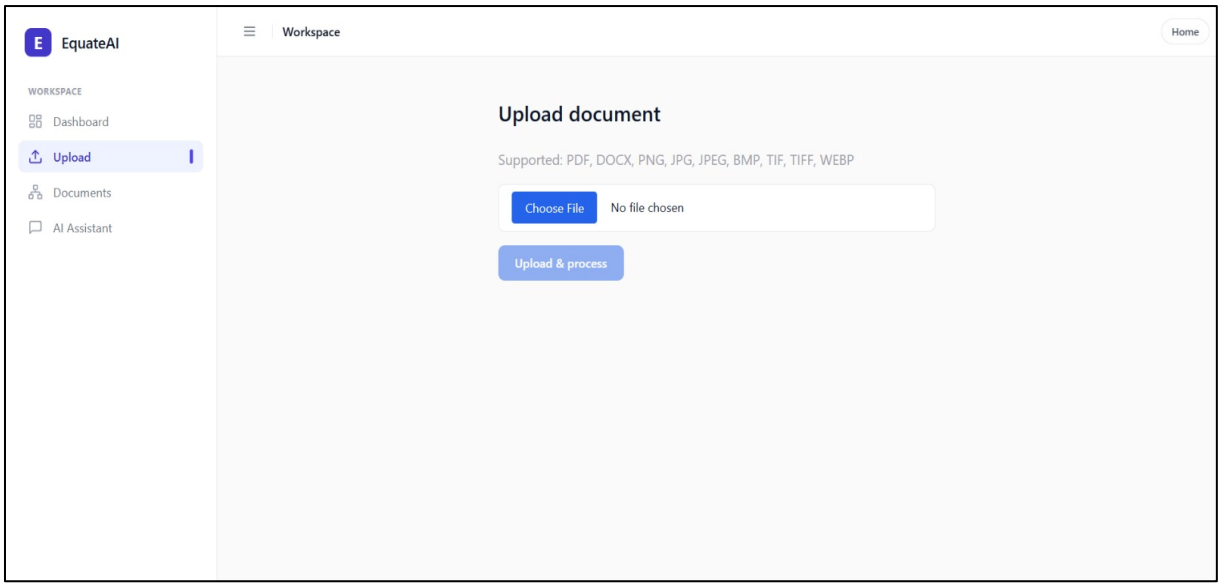

Figure 6.2: Document Upload Page

6.3 Document Analysis and Equation Extraction:

Once a document is processed, the system displays extracted equations along with metadata such as number of pages, equations, and undefined variables, as shown in Figure 6.3.

Each equation is presented along with its variables and possible issues such as conflicts or multiple definitions. The system also highlights inconsistencies and provides quick access to explanation features.

.

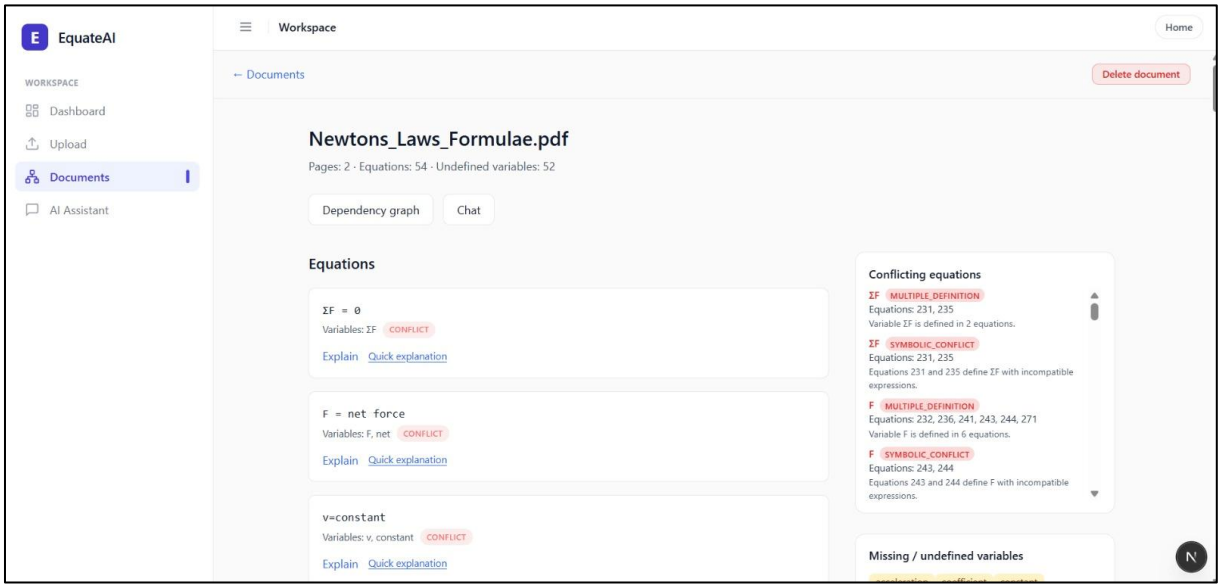

Figure 6.3: Extracted Equations and Analysis

6.4 AI-Based Equation Explanation:

The system provides AI-generated explanations for selected equations, as illustrated in Figure 6.4. When a user selects an equation, the system generates a human-readable explanation using a local language model.

This feature helps users understand the meaning, variables, and context of equations without requiring manual interpretation. It improves accessibility, especially for complex scientific expressions.

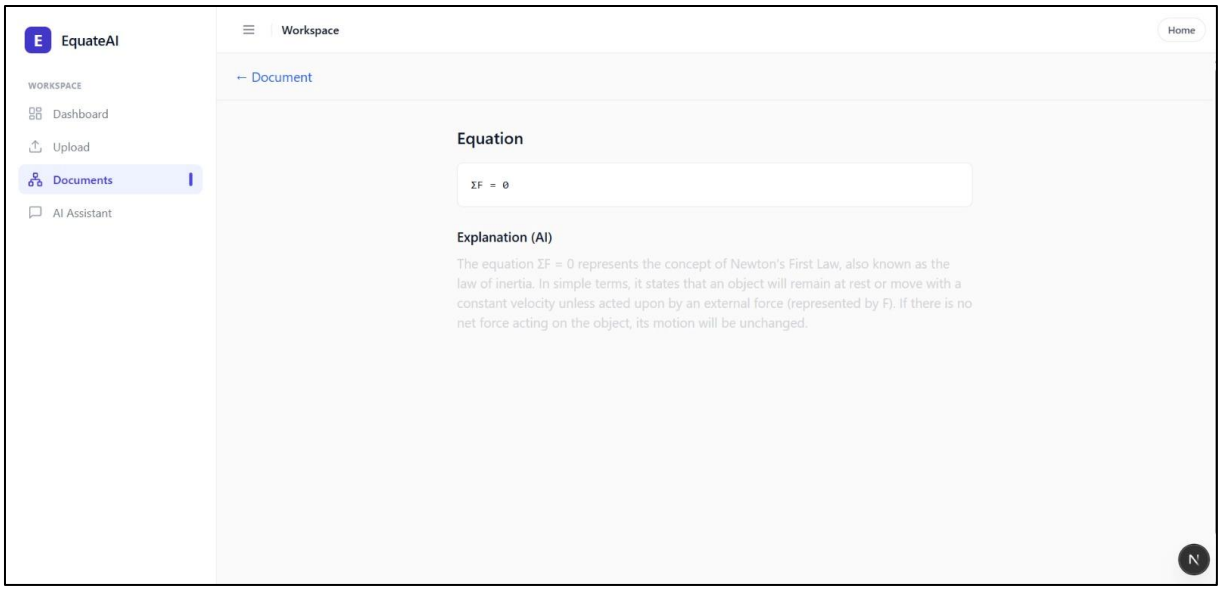

Figure 6.4: AI-Based Equation Explanation

6.5 Dependency Graph Visualization:

The dependency graph, shown in Figure 6.5, visually represents relationships between variables and equations. Nodes represent variables, equations, concepts, and constants, while edges indicate dependencies.

Users can interact with the graph, apply filters, and inspect node details. This visualization helps in understanding how variables are connected and how equations depend on one another.

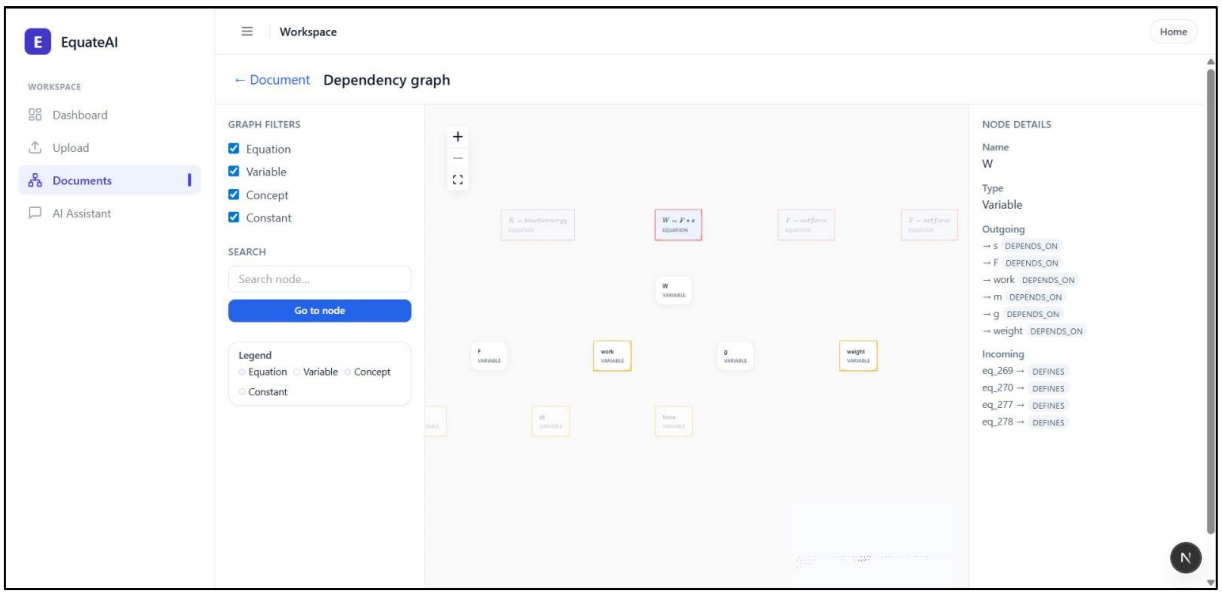

Figure 6.5: Dependency Graph Visualization

6.6.AI Assistant (Chat Interface):

AI Assistant module, shown in Figure 6.6, allows users to interact with the system using natural language queries. Users can ask questions related to equations or request explanations directly.

For example, when a user asks for the equation of force, the system responds with the formula and its explanation. This feature enhances user engagement and provides an intuitive way to access knowledge extracted from documents.

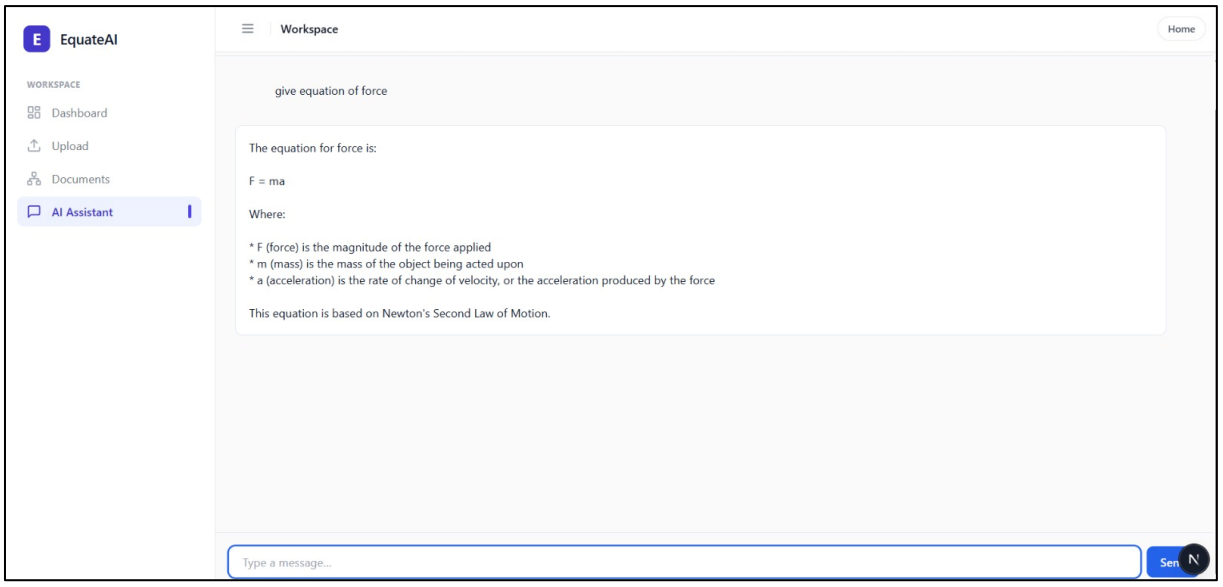

Figure 6.6: AI Assistant Chat Interface

CHAPTER 7
CONCLUSION AND FUTURE WORK

The development of **EquateAI: AI-Assisted Scientific Equation Parsing, Graph Construction, and Document-Centric Reasoning** represents a significant advancement in the way scientific documents are analyzed and interpreted. The system was designed to provide an intelligent, user-friendly, and efficient platform for processing complex documents containing mathematical expressions, extracting structured knowledge, and assisting users through AI-driven explanations.

By combining document processing, symbolic reasoning, graph-based analysis, and local language model integration, the system transforms unstructured textual data into meaningful, structured insights. This approach not only improves accessibility to mathematical content but also supports deeper understanding through visualization and contextual interaction.

7.1 Conclusion:

The successful implementation of EquateAI has resulted in several key benefits:

- **Centralized Knowledge Representation:** All extracted information from documents, including equations, variables, dependencies, conflicts, and analysis results, is stored in a structured and centralized format. This reduces redundancy and ensures consistency in data representation.
- **Enhanced Interpretability:** The system provides structured outputs such as Abstract Syntax Trees (ASTs), dependency graphs, and knowledge graphs, enabling users to visually explore relationships between variables and equations.
- **AI-Assisted Understanding:** Integration of local language models enables equation explanation and document-aware chat functionality, helping users better understand complex mathematical expressions and their context.
- **Scalable and Modular Architecture:** The system is designed with a modular backend and scalable frontend, allowing easy extension of features, support for new document types, and integration of additional analytical capabilities

7.2 Limitations:

- **Complex Equation Handling:** The current parsing and detection mechanisms may struggle with highly complex or non-standard mathematical notations, especially when extracted from low-quality documents or images.
- **OCR Accuracy Constraints:** When processing image-based documents, the accuracy of text extraction depends on OCR quality, which may introduce noise or errors in the extracted content.
- **Dependency on Local Models:** The AI explanation and chat features rely on locally running language models, which may have limitations in reasoning depth compared to large cloud-based models.
- **Performance on Large Documents:** Processing very large documents with numerous equations may increase execution time and resource usage, affecting system responsiveness.
- **Limited Evaluation Benchmarks:** The system is primarily evaluated based on functionality and usability rather than standardized datasets and quantitative benchmarks.

7.3 Future Work:

While EquateAI provides a strong foundation for document-centric equation analysis, several enhancements can further improve its capabilities:

- **Advanced Mathematical Parsing:** Improve parsing algorithms to support more complex equations, multi-line expressions, and domain-specific mathematical formats.
- **Enhanced OCR and Document Understanding:** Integrate more advanced OCR and layout-aware models to improve extraction accuracy from scanned or low-quality documents.
- **Cloud Deployment and Scalability:** Extend the system for cloud-based deployment using containerization (Docker) and scalable infrastructure to handle larger workloads and multiple users.
- **Improved Graph Visualization:** Enhance graph layouts, interactivity, and visualization techniques to provide more intuitive exploration of relationships.

REFERENCES

1. Extracting Context of Math Formulae Contained inside Scientific Documents
2. A Math Formula Extraction and Evaluation Framework for PDF Documents
3. FastAPI – Official Documentation, FastAPI Docs
4. Next.js – Official Documentation, Next.js Docs
5. React – Official Documentation, React Docs
6. SQLAlchemy – Official Documentation, SQLAlchemy Docs
7. SymPy – Official Documentation, SymPy Docs
8. NetworkX – Official Documentation, NetworkX Docs
9. Ollama – Official Website, Ollama Docs
10. PyMuPDF – Documentation, PyMuPDF Docs
11. pytesseract – Documentation, pytesseract Docs
12. python-docx – Documentation, python-docx Docs
13. Git – Official Website, Git Docs

APPENDIX

APPENDIX -A

Programme Educational Objectives (PEOs):

PEO 1: Graduates of the program are adequately prepared to be employed in IT industries and Public Sector companies by forecasting a logical and practical approach to problem solving that would prepare them to function effectively as skilled computer engineers.

PEO 2: To impart students with solid foundation in mathematics, computing and core engineering fundamentals so as to help them to excel in their professional career or higher education.

PEO 3: To promote lifelong learning by encouraging research and an attitude to apply the basic theories learnt during their graduation, leading to creativity and productivity in their respective fields.

PEO 4: To inculcate students with leadership qualities, communication skills and ethical behavior as IT professionals that can lead to positive impact of technology on society.

APPENDIX B

Programme Outcomes (POs):

PO 1: Engineering knowledge
Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO 2: Problem analysis
Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences**.**

PO 3: Design/development of solutions
Design solutions for complex engineering problems and design system components or processes that meet specified needs with consideration for public health, safety, and societal factors.

PO 4: Conduct investigations of complex problems
Use research-based knowledge and methods including experiment design, data analysis, and interpretation to provide valid conclusions.

PO 5: Modern tool usage
Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools with understanding of limitations.

PO 6: The engineer and society
Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal, and cultural issues.

PO 7: Environment and sustainability
Understand environmental impact and demonstrate knowledge of sustainable development.

PO 8: Ethics
Apply ethical principles and commit to professional responsibilities and norms.

PO 9: Individual and team work
Function effectively as an individual and as a member or leader in teams.

PO 10: Communication
Communicate effectively through reports, presentations, and clear instructions.

PO 11: Project management and finance
Apply engineering and management principles in project execution and teamwork.

PO 12: Life-long learning
Recognize and engage in independent and lifelong learning.

APPENDIX C

Program Specific Outcomes (PSOs):

PSO 1: Ability to understand and apply concepts of mathematics, basic sciences, and computing in solving real-world problems.

PSO 2: Ability to design and develop computer-based systems in areas like algorithms, networking, web design, and data analytics using software engineering practices.

PSO 3: Ability to imbibe ethical and professional skills required for teamwork and leadership.

APPENDIX D

Articulation Matrix for the Project (Scientific Equation Parser & Analysis Tool):

Articulation Scale:

- 3 – High
- 2 – Moderate
- 1 – Low

Project Component	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
Problem analysis & requirements study	3	3	2	2	2	1	1	1	1	2	1	2	3	1	1
System design & architecture	2	2	3	2	2	1	1	1	2	2	2	1	2	3	2
Document processing & equation parsing	3	3	2	3	2	1	1	1	1	1	1	2	3	2	1
Semantic analysis & graph generation	3	3	3	3	2	1	1	1	1	1	1	2	3	2	1
AI explanation & chat module	2	2	2	2	3	1	1	1	2	3	1	2	2	3	2
Database design & retrieval system	2	2	2	2	3	1	1	1	2	2	2	1	2	3	2
Teamwork, reporting & presentation	1	1	1	1	1	1	1	2	3	3	2	2	1	1	3

Note:

The **EquateAI project strongly supports PO1, PO2, PO3, PO4, PO5, PO10, and PSO2** as it involves:

- Advanced problem analysis of unstructured scientific data
- Mathematical and symbolic reasoning
- Graph-based modeling and knowledge representation
- Use of modern tools (FastAPI, NLP, LLMs, OCR)
- AI-assisted explanation and technical communication

APPENDIX E

Sustainable Development Goals (SDGs) with Articulation Matrix

Relevant SDGs for EquateAI

- **SDG 4: Quality Education**
- **SDG 9: Industry, Innovation and Infrastructure**
- **SDG 12: Responsible Consumption and Production**
- **SDG 16: Peace, Justice and Strong Institutions**

SDG Mapping Table:

SDG	Relevance to EquateAI	Level
SDG 4	Enhances learning by simplifying understanding of complex scientific equations	3
SDG 9	Promotes innovation in AI-driven document analysis and knowledge extraction	3
SDG 12	Reduces manual effort and improves efficient knowledge utilization	2
SDG 16	Improves transparency and traceability in scientific knowledge interpretation	2

Explanation:

EquateAI supports **SDG 4 and SDG 9** by making scientific knowledge easier to understand using AI. It also contributes to **SDG 12** through efficient knowledge processing and **SDG 16** by improving transparency and structured representation of information.